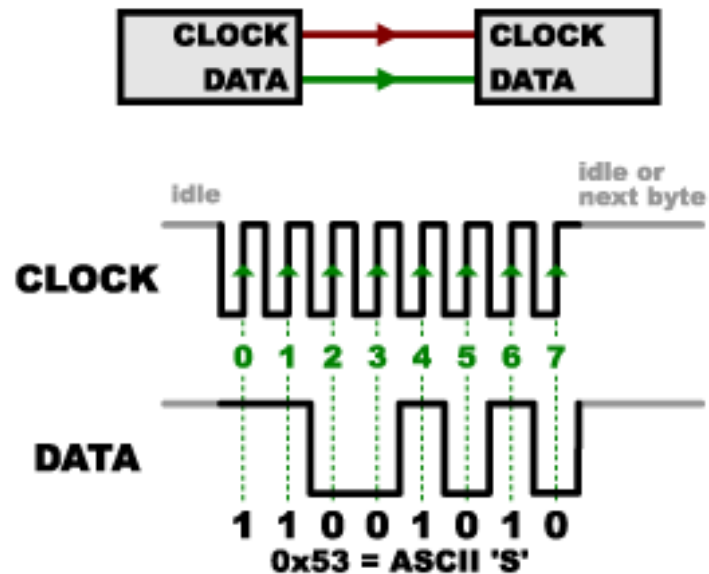


Comunicação serial



Definição

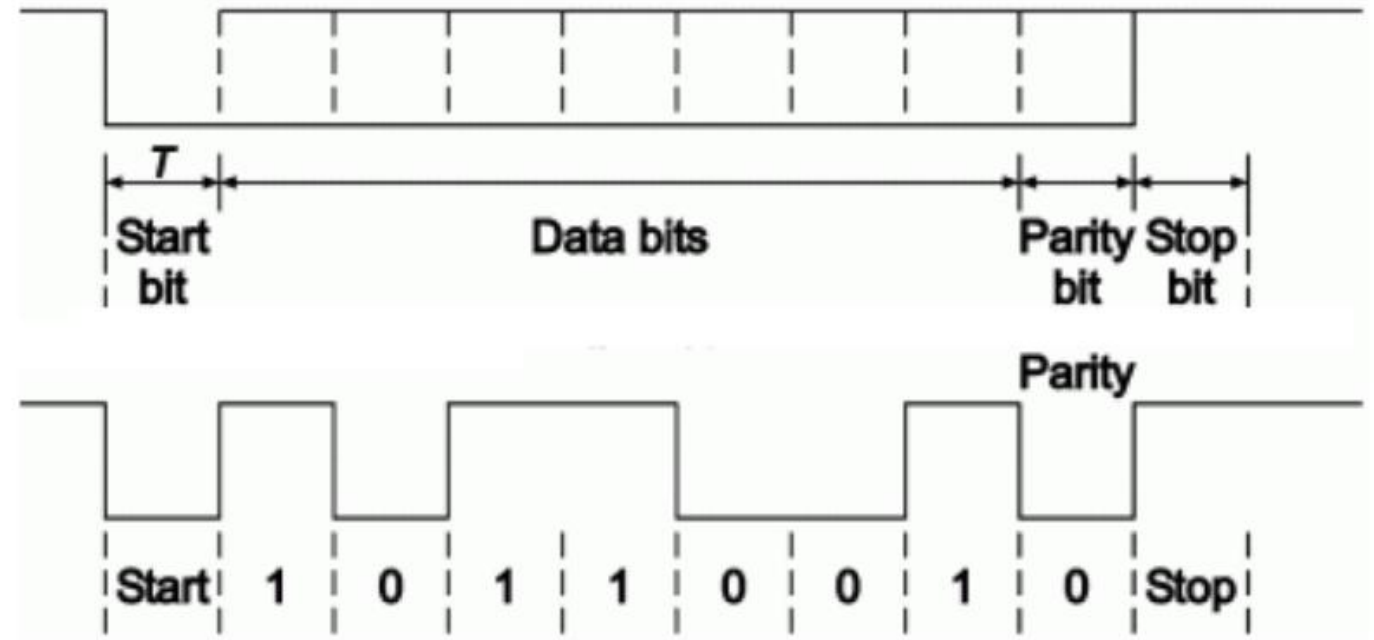
- ❖ A comunicação serial é o processo de enviar e receber dados um bit de cada vez, sequencialmente, em um canal de comunicação ou barramento.
- ❖ Ela pode ser de dois tipos:
- ❖ Síncrona: quando transmite junto aos dados um sinal de clock.
- ❖ Assíncrona: quando transmite sem o sinal de clock.

UART

- UART significa Universal Asynchronous Receiver Transmitter.
- É um protocolo de comunicação serial que permite a troca de dados entre dois dispositivos.
- A UART é comumente usada em aplicações como comunicação de computador para periférico, comunicação entre microcontroladores e comunicação com dispositivos embarcados.

UART

O padrão UART é um exemplo de comunicação serial assíncrona, a temporização da transmissão dos dados é definida pela velocidade de transmissão em bits por segundo (bps)

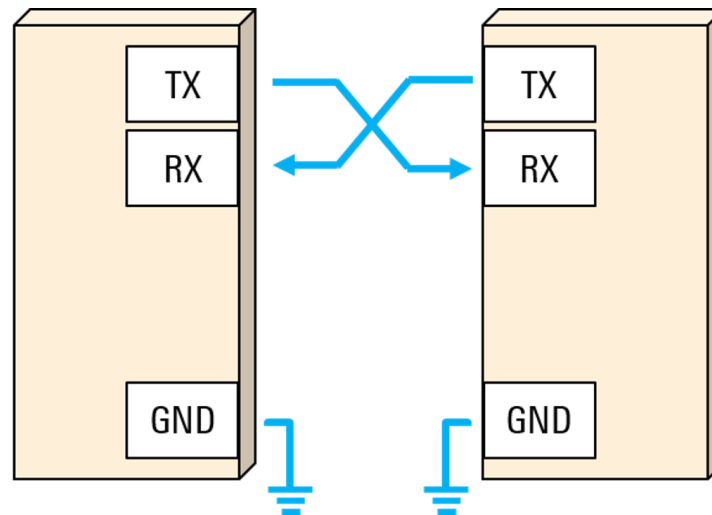


Forma de onda de um sinal serial assíncrono.

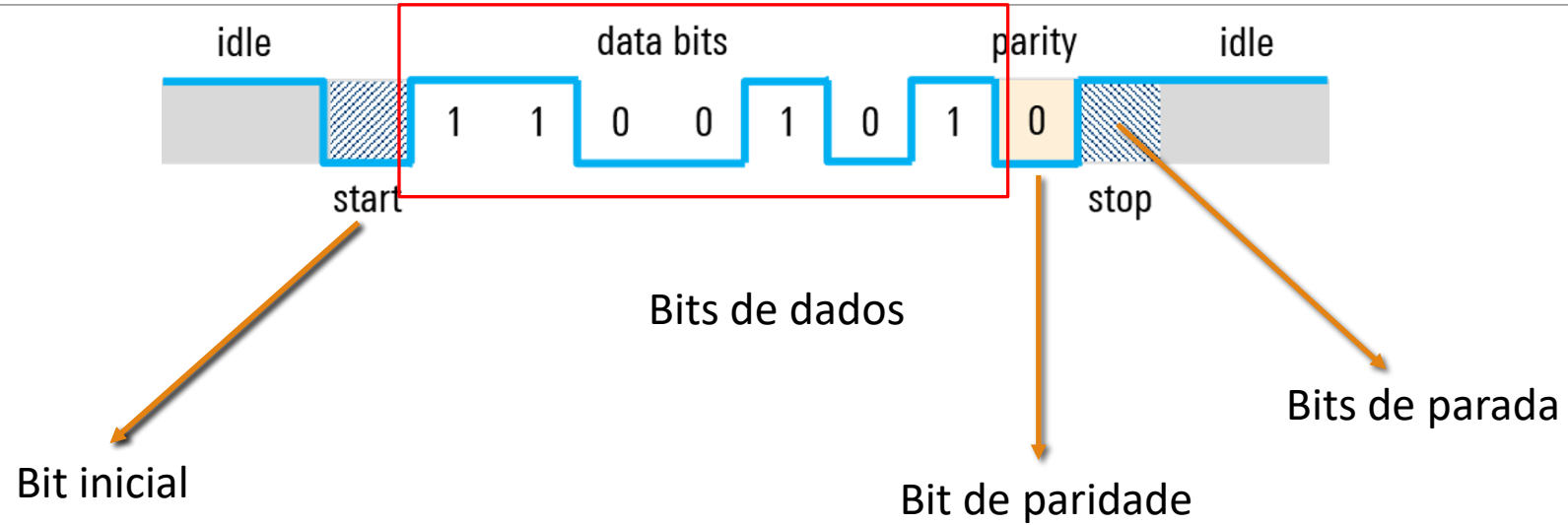
Características UART

- ❖ Comunicação Assíncrona.
- ❖ Configurações Principais:
 - ❖ Baud Rate (ex.: 9600, 115200)
 - ❖ Bits de Dados (normalmente 8)
 - ❖ Paridade (Par, Ímpar ou Nenhuma)
 - ❖ Bits de Parada (geralmente 1 ou 2)

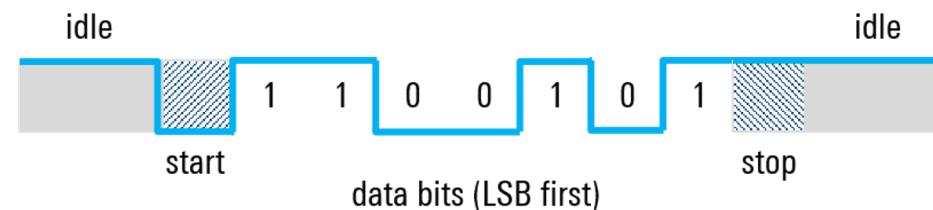
-
- ❖ Para estabelecer uma comunicação serial é necessário então que os dois dispositivos estejam no mesmo potencial, ou seja, o negativo da alimentação dos dois dispositivos deve estar conectado.
 - ❖ É necessário também que a saída de dados do transmissor seja conectada a entrada de dados do receptor.



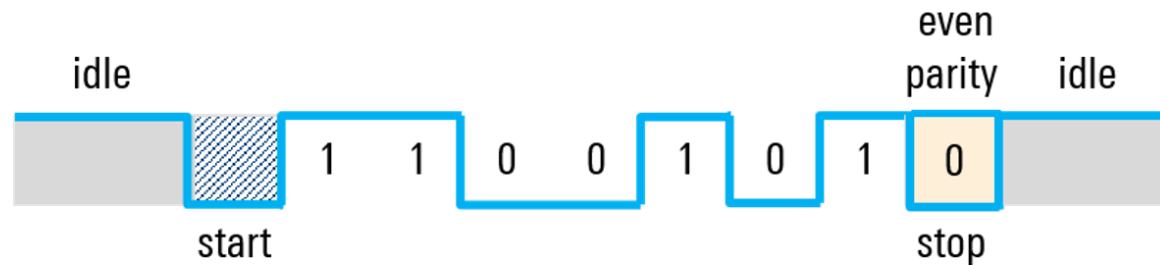
Formato do frame



-
- ❖ O bit inicial é uma transição do estado inativo para um estado baixo, imediatamente seguido pelos bits de dados do usuário.
 - ❖ O bit final indica o fim dos dados do usuário. O bit de parada é uma transição de volta para o estado alto ou inativo.
 - ❖ Os bits de dados são dados de usuário. Pode haver de 5 a 9 bits de dados.
 - ❖ O mais comum é usar 8 bits.
 - ❖ Esses bits de dados geralmente são transmitidos com o bit menos significativo primeiro.



-
- ❖ O bit de paridade é inserido entre o fim dos bits de dados e o bit final. O valor do bit de paridade depende do tipo de paridade sendo utilizado (par ou ímpar):
 - ❖ Na paridade par, esse bit é definido de modo que o número total de “UNS” no frame seja par.
 - ❖ Na paridade ímpar, esse bit é definido de modo que o número total de “UNS” no frame seja ímpar.



Transmitindo Dados via UART

```
/**
 * @brief Sends an amount of data in blocking mode.
 * @note When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01),
 * the sent data is handled as a set of u16. In this case, Size must indicate the number
 * of u16 provided through pData.
 * @param huart Pointer to a UART_HandleTypeDef structure that contains
 * the configuration information for the specified UART module.
 * @param pData Pointer to data buffer (u8 or u16 data elements).
 * @param Size Amount of data elements (u8 or u16) to be sent
 * @param Timeout Timeout duration
 * @retval HAL status
 */
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, const uint8_t *pData, uint16_t Size, uint32_t Timeout)
{
    /* ... */
}
```

```
// Usando sprintf para formatar texto com número inteiro
sprintf(msg, "RPM Atual = %d\r\n", frequencia);
HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
```

Recebendo Dados via UART

```
/**
 * @brief  Receives an amount of data in blocking mode.
 * @note   When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01),
 *         the received data is handled as a set of u16. In this case, Size must indicate the number
 *         of u16 available through pData.
 * @param  huart Pointer to a UART_HandleTypeDef structure that contains
 *         the configuration information for the specified UART module.
 * @param  pData Pointer to data buffer (u8 or u16 data elements).
 * @param  Size  Amount of data elements (u8 or u16) to be received.
 * @param  Timeout Timeout duration
 * @retval HAL status
 */
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
{
```

```
// Recebe até 20 caracteres
char rxData[20];
```

```
HAL_StatusTypeDef status = HAL_UART_Receive(&huart1, (uint8_t *)rxData, sizeof(rxData), HAL_MAX_DELAY);
```

```
sprintf(msg, "Porc TPS2 = %.2f\r\n", porcTPS2); //Exemplo de uso do Sprintf para converter ponto flutuante
//em string
HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 50);
```

Para que a função `sprintf` formate corretamente números de ponto flutuante (como `%f`) na STM32CubeIDE, é necessário habilitar o suporte a ponto flutuante na configuração do projeto. Por padrão, a biblioteca `newlib-nano` usada nos projetos da STM32CubeIDE tem essa funcionalidade desabilitada para economizar memória, já que o suporte a ponto flutuante consome mais recursos.

Para resolver isso, siga os passos abaixo:

1. No **Project Explorer**, clique com o botão direito no seu projeto e selecione **Properties**.
2. Na janela de propriedades, navegue para **C/C++ Build -> Settings**.
3. Vá para a aba **Tool Settings** e selecione **MCU Settings**.
4. Marque a caixa ao lado de **Use float with printf from newlib-nano**.
5. Clique em **Apply and Close**.

